

Diretivas

📁 Caminho: Projeto03_Diretivas/src/app/app.ts

```
TS app.ts  X  </> app.html
Projeto03_Diretivas > src > app > TS app.ts > ...
1  import { Component, signal } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    standalone: true,    // Indica que o componente é standalone
6    templateUrl: './app.html',
7    styleUrls: ['./app.css']
8  })
9  export class App {
10    protected exibir = signal(false);
11
12    // 1. Variável de sim/não
13    ligado = signal(false);
14
15    // 2. Lista simples
16    frutas = signal(['Maçã', 'Banana', 'Uva']);
17
18  }
19  Ctrl+L for Agent
```

ligado = signal(false);

Este trecho declara um sinal booleano inicializado como **false**. Sob a ótica de engenharia, este atributo funciona como uma **variável de estado binário**.

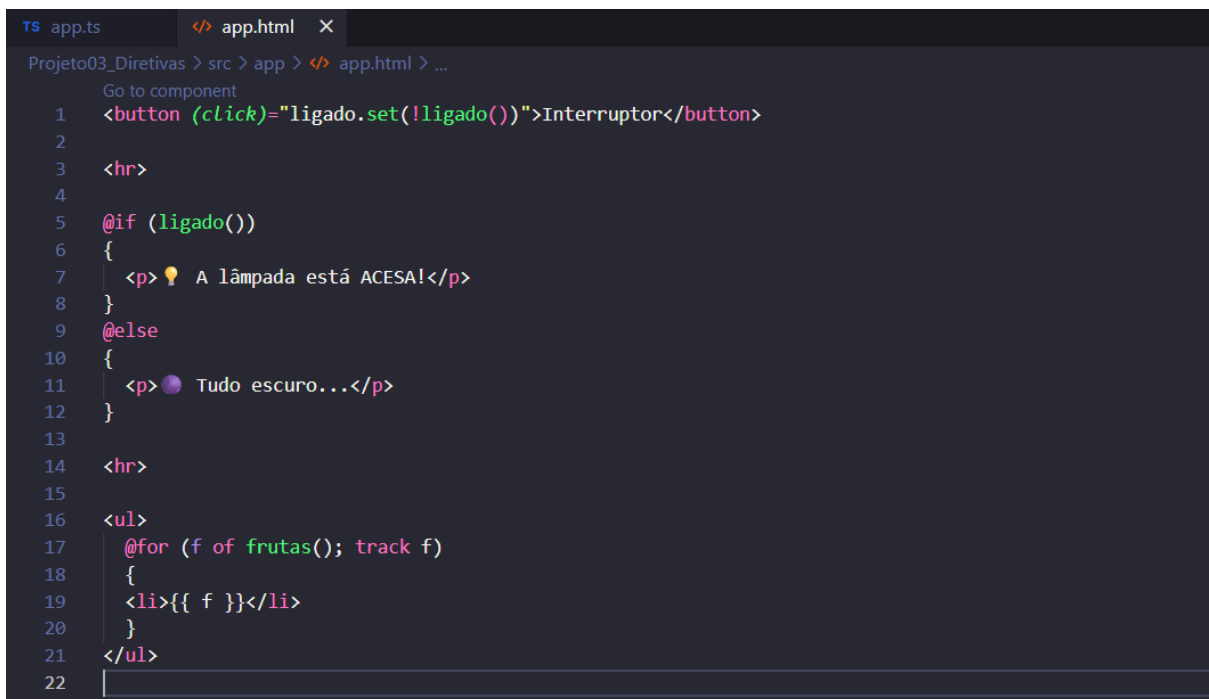
- **Função:** Ele é utilizado pelo front-end para tomar decisões lógicas (ex: mostrar ou esconder um elemento).
- **Mecânica:** Ao contrário de uma variável primitiva, o Signal atua como um "ponteiro reativo". Quando o valor é alterado, o Angular identifica a mudança e atualiza apenas os nós do DOM que dependem deste estado específico.

frutas = signal(['Maçã', 'Banana', 'Uva']);

Este comando inicializa um sinal contendo um **Array de Strings**.

- **Função:** Representa uma coleção de dados linear. No contexto de um sistema escalável, este array simula o retorno de uma consulta a um banco de dados ou serviço externo.
- **Mecânica:** O Signal encapsula o Array, permitindo que a diretiva de repetição no HTML (**@for**) mantenha uma sincronia exata com a lista presente no TypeScript. Se um item for adicionado ou removido desta lista no "back-end" do componente, a interface refletirá a alteração instantaneamente.

📁 **Caminho:** Projeto03_Diretivas/src/app/app.html



```
TS app.ts  </> app.html  X
Projeto03_Diretivas > src > app > </> app.html > ...
Go to component
1  <button (click)="ligado.set(!ligado())">Interruptor</button>
2
3  <hr>
4
5  @if (ligado())
6  {
7    <p>💡 A lâmpada está ACESA!</p>
8  }
9  @else
10 {
11   <p>💡 Tudo escuro...</p>
12 }
13
14 <hr>
15
16 <ul>
17   @for (f of frutas(); track f)
18   {
19     <li>{{ f }}</li>
20   }
21 </ul>
22
```

O arquivo HTML utiliza as novas diretivas de controle de fluxo do Angular para reagir em tempo real aos estados definidos no arquivo de lógica (**app.ts**).

1. Controle Condicional (**@if** e **@else**)

O bloco **@if** atua como um filtro estrutural que escuta o sinal **ligado()**.

- **Mecânica de Recebimento:** Ao ler **ligado()**, o HTML recebe o valor booleano vindo do "back-end" do componente.

- **Ação:** Se o valor for `true`, o Angular injeta o elemento `<p>` da lâmpada acesa no DOM. Se for `false`, o bloco `@else` assume o controle, garantindo que o navegador remova fisicamente o conteúdo anterior e exiba a mensagem "Tudo escuro..."

2. Iteração de Dados e o Papel do `track` (`@for`)

O bloco `@for` é o responsável por transformar a coleção de dados do Signal `frutas()` em elementos visuais de lista (`<li>`).

- **O "ID" da Renderização (`track`):** Na engenharia de software, a performance é ditada pela eficiência. O comando `track f` funciona como o **ID (Chave Primária)** de cada item.
- **Por que o `track` é vital?** Ele informa ao motor do Angular exatamente qual item da lista é qual. Se um item mudar de posição, o Angular usa esse "ID" para apenas mover o elemento no HTML, em vez de destruir e recriar a lista inteira. Isso economiza processamento e memória do dispositivo.